LAPORAN UJIAN TENGAH SEMESTER

PEMROGRAMAN MULTIPLATFORM

PENGEMBANGAN APLIKASI DOMPETKU BERBASIS FLUTTER DENGAN ASYNCHRONOUS PROGRAMMING, SQLITE, DAN SHAREDPREFERENCES

Disusun Oleh:

MUHAMAD KEVIN DWI ARISANDI

NIM: 240605110252

Kelas: [KELAS]

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM
MALANG

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang dan Gambaran Aplikasi

Pengelolaan keuangan pribadi harian merupakan salah satu aspek penting dalam mewujudkan kedisiplinan finansial, khususnya bagi mahasiswa dan individu produktif. Dalam kehidupan sehari-hari, arus kas masuk (pemasukan) dan keluar (pengeluaran) sering kali tidak terpantau secara sistematis karena transaksi kecil yang berulang tidak dicatat secara disiplin. Hal ini sering menimbulkan ketidakseimbangan anggaran, pemborosan, dan kesulitan dalam mengevaluasi riwayat pengeluaran bulanan.

Sebagian besar aplikasi keuangan komersial modern yang beredar saat ini mewajibkan pengguna untuk melakukan registrasi akun online, menghubungkan layanan cloud, atau selalu berada dalam jaringan internet aktif. Pendekatan berbasis cloud tersebut kerap menghadirkan kendala ketergantungan koneksi jaringan (network latency), potensi kebocoran privasi data keuangan pribadi ke server pihak ketiga, serta keharusan menyiapkan backend server yang kompleks seperti Firebase atau MySQL/XAMPP.

Untuk menjawab tantangan tersebut, dikembangkanlah aplikasi mobile bernama DompetKu berbasis framework Flutter dan bahasa pemrograman Dart. DompetKu dirancang dengan arsitektur mandiri (standalone) yang berfokus pada pendekatan offline-first, di mana seluruh operasi data keuangan dieksekusi secara lokal di perangkat tanpa memerlukan server cloud, Firebase, maupun backend XAMPP. Aplikasi ini mengintegrasikan dua pilar utama dalam kurikulum Pemrograman Multiplatform:

1. Asynchronous Programming secara komprehensif menggunakan Future, async, dan await untuk menjamin bahwa proses inisialisasi basis data, transaksi finansial, kalkulasi ringkasan keuangan, serta penyimpanan preferensi tidak memblokir antarmuka pengguna (UI thread).

2. Dua teknik Data Persistence lokal yang berbeda karakteristik dan fungsinya, yaitu SQLite (menggunakan package sqflite dan path) untuk penyimpanan data transaksional relasional yang kompleks, serta SharedPreferences untuk penyimpanan data konfigurasi ringan berupa preferensi tema antarmuka dan batas anggaran bulanan.

Dengan perpaduan antarmuka modern yang responsif berbasis Material Design 3, validasi transaksi yang ketat, dan persistensi lokal yang teruji, DompetKu menghadirkan solusi pencatatan kas harian yang cepat, aman, privat, serta konsisten di berbagai kondisi perangkat.

1.2 Fitur Aplikasi

Aplikasi DompetKu memiliki serangkaian fitur fungsional utama yang dirancang untuk memenuhi seluruh kebutuhan operasional keuangan pengguna:

a. Dashboard Keuangan Terintegrasi: Menampilkan kartu saldo saat ini (all-time balance), total akumulasi pemasukan bulan berjalan, total pengeluaran bulan berjalan, ringkasan penggunaan anggaran bulanan (persentase visual dan nominal sisa), serta lima daftar transaksi terkini yang diperbarui secara realtime langsung dari query agregasi SQL.

b. Manajemen Transaksi Finansial (CRUD): Menyediakan formulir terintegrasi untuk menambah transaksi pemasukan atau pengeluaran, memilih kategori yang terasosiasi, mengisi nominal tervalidasi (angka positif terbatas), memilih tanggal transaksi interaktif, mengisi deskripsi/catatan, mengedit transaksi yang ada, serta menghapus transaksi dengan dialog konfirmasi keamanan.

c. Kategori Transaksi Relasional: Menyediakan kategori awal terstruktur (Gaji, Bonus, Investasi, Penjualan untuk pemasukan; Makanan & Minuman, Transportasi, Pendidikan, Belanja Harian, Hiburan, Kesehatan, Tagihan, Lainnya untuk pengeluaran) yang tersimpan secara relasional di SQLite dengan ikon dan identitas visual.

d. Riwayat dan Filter Transaksi: Halaman daftar transaksi lengkap yang dilengkapi fitur penyaringan instan multi-dimensi berdasarkan tipe transaksi (Semua, Pemasukan, Pengeluaran), kategori transaksi, serta periode bulan kalender.

e. Batas Anggaran Bulanan (Monthly Budgeting): Fitur penetapan batas maksimal pengeluaran bulanan yang disimpan secara persisten di SharedPreferences, dilengkapi bilah kemajuan (LinearProgressIndicator) yang menghitung persentase beban pengeluaran terhadap pagu anggaran secara otomatis.

f. Pengaturan Tema Antarmuka (Dark / Light Mode): Pengguna dapat beralih antara tema terang (Light Mode) dan tema gelap (Dark Mode) secara instan melalui ikon AppBar, di mana preferensi tema disimpan secara persisten di SharedPreferences dan dipulihkan kembali saat aplikasi diluncurkan ulang.

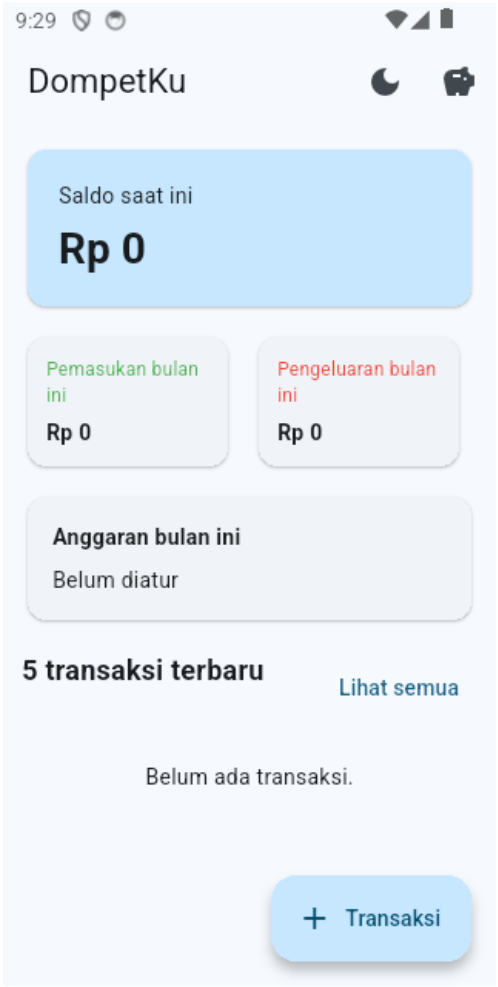
BAB II

PEMBAHASAN

2.1 Desain Antarmuka Pengguna

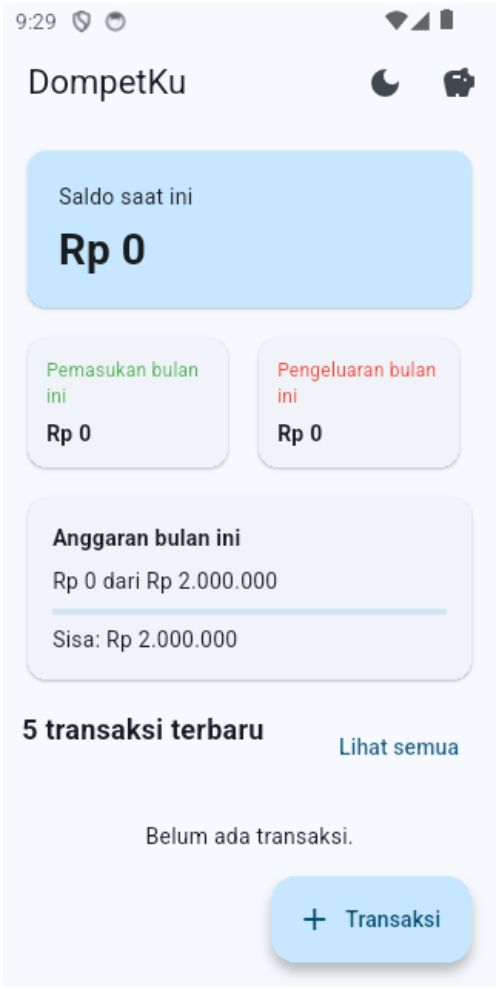
Desain antarmuka pengguna (UI) pada aplikasi DompetKu mengadopsi prinsip desain modern Material Design 3 yang mengutamakan kesederhanaan, keterbacaan tipografi, hierarki visual yang jelas, serta kenyamanan navigasi. Antarmuka dirancang responsif menggunakan kombinasi widget ListView, Wrap, SingleChildScrollView, Card, dan LayoutBuilder, sehingga tata letak secara adaptif menyesuaikan diri pada berbagai dimensi layar perangkat Android (mulai dari layar kompak 320 logical pixel hingga layar standar 412 logical pixel) tanpa menimbulkan kesalahan RenderFlex Overflow.

Berikut adalah dokumentasi tangkapan layar asli (screenshot) hasil pengujian langsung pada Android Emulator yang mendemonstrasikan seluruh alur antarmuka dan interaksi pengguna dalam aplikasi DompetKu:



Gambar 2.1: Tampilan Dashboard Awal DompetKu (Kondisi Bersih / Fresh Install)

Tangkapan layar di atas menunjukkan kondisi awal aplikasi DompetKu saat pertama kali dipasang dan dibuka. Saldo saat ini, pemasukan bulan ini, dan pengeluaran bulan ini bernilai Rp 0 karena belum ada transaksi yang dicatat. Kartu anggaran bulanan menampilkan status 'Belum diatur' yang mengundang pengguna untuk menetapkan batas anggaran. Pada bagian bawah, daftar transaksi menampilkan empty state informatif yang menyatakan bahwa belum ada data transaksi yang tersimpan di SQLite.



Gambar 2.2: Penetapan Batas Anggaran Bulanan Melalui Dialog Modal

Pengguna membuka dialog modal anggaran dengan menekan ikon tabungan pada AppBar. Dialog ini menyediakan input numerik untuk memasukkan batas pengeluaran bulanan dalam mata uang Rupiah. Pada pengujian ini, pengguna menetapkan batas anggaran sebesar Rp 3.000.000. Nilai ini divalidasi dan disimpan secara persisten ke SharedPreferences melalui BudgetPreferenceService, sehingga kartu dashboard langsung memperbarui informasi batas anggaran dan sisa pagu belanja.



Gambar 2.3: Formulir Penambahan Transaksi Keuangan

Formulir transaksi dapat diakses melalui Floating Action Button (+ Transaksi) pada dashboard. Formulir ini dilengkapi tombol pilihan tipe (Pengeluaran dan Pemasukan), input nominal tervalidasi yang hanya menerima angka bulat positif, pemilih kategori dinamis yang memfilter kategori berdasarkan tipe yang dipilih, date picker interaktif untuk memilih tanggal transaksi, serta kolom catatan deskriptif opsional.



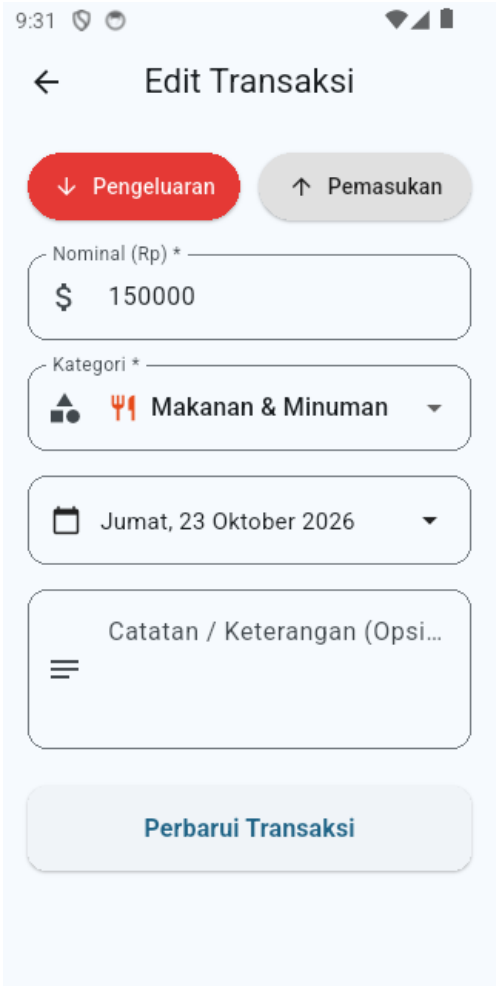
Gambar 2.4: Pembaruan Dashboard Setelah Pencatatan Transaksi Pemasukan

Setelah pengguna menyimpan transaksi pemasukan berupa 'Gaji' sebesar Rp 5.000.000, fungsi asinkron _load() pada DashboardScreen dipanggil kembali. Ringkasan keuangan secara otomatis dihitung ulang melalui query agregasi SQL SQLite, sehingga kartu Saldo Saat Ini dan kartu Pemasukan Bulan Ini seketika berubah menjadi Rp 5.000.000, serta item transaksi langsung muncul pada daftar 5 transaksi terbaru.



Gambar 2.5: Dashboard dengan Akumulasi Pemasukan, Pengeluaran, dan Anggaran

Pada pengujian berikutnya, pengguna menambahkan transaksi pengeluaran untuk kategori 'Makanan & Minuman' sebesar Rp 150.000. Saldo saat ini secara otomatis terkalkulasi menjadi Rp 4.850.000 (Rp 5.000.000 - Rp 150.000). Bilah LinearProgressIndicator pada kartu anggaran bulanan secara visual menampilkan persentase penggunaan dana sebesar 5% dari pagu Rp 3.000.000 dengan sisa anggaran Rp 2.850.000.



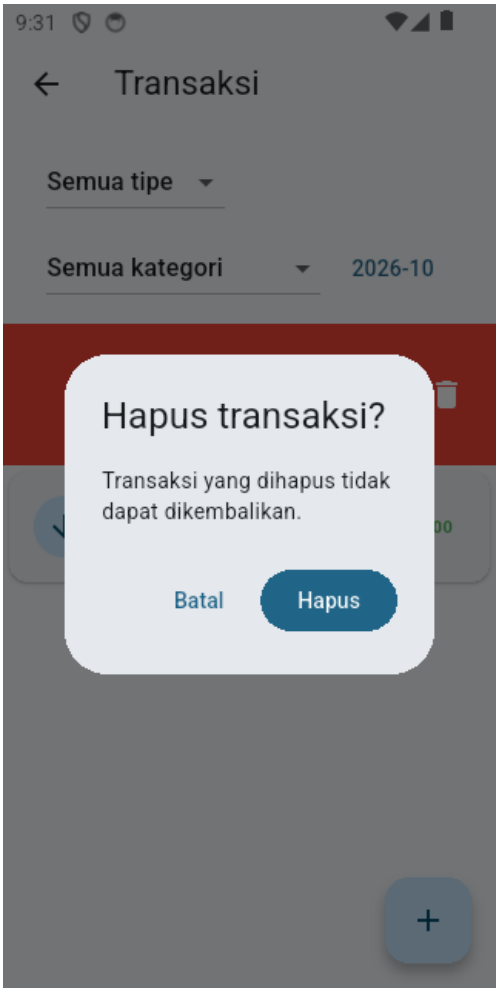
Gambar 2.6: Formulir Pengeditan Transaksi Eksisting

Ketika salah satu transaksi pada daftar diketuk, formulir pengeditan terbuka dengan data yang telah terisi (pre-populated) sesuai dengan record yang tersimpan di SQLite. Pengguna dapat mengubah nominal, kategori, tanggal transaksi, maupun catatan keterangan. Pengujian ini memastikan bahwa atribut id dan created_at tetap terjaga secara konsisten saat proses update dilakukan.



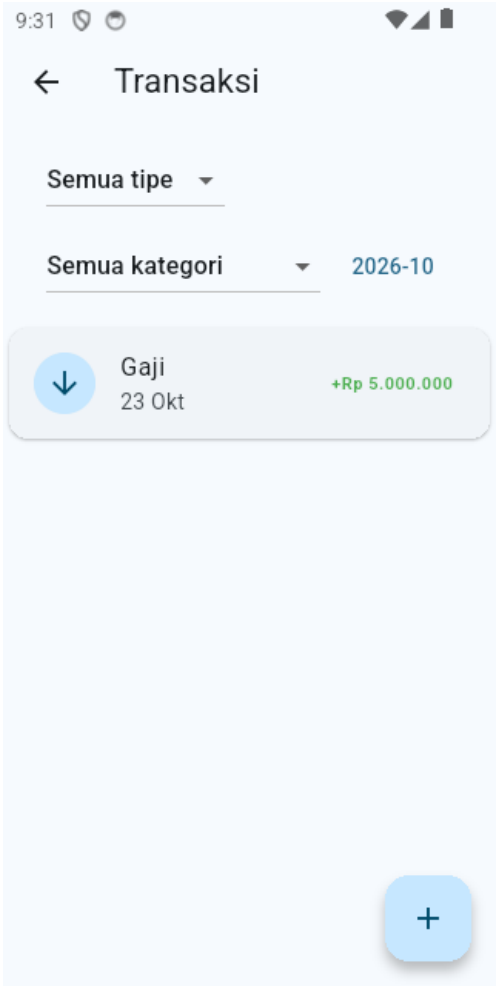
Gambar 2.7: Daftar Transaksi Setelah Pembaruan Data Berhasil

Halaman daftar transaksi lengkap yang diakses melalui tombol 'Lihat semua' menampilkan daftar seluruh riwayat pencatatan. Pada tangkapan layar ini, terlihat hasil pembaruan data transaksi yang telah diedit. Halaman ini juga menyediakan filter dropdown untuk memfilter transaksi berdasarkan tipe (Semua/Pemasukan/Pengeluaran), kategori, serta pemilih bulan.



Gambar 2.8: Dialog Konfirmasi Penghapusan Transaksi

Untuk mencegah hilangnya data akibat ketidaksengajaan pengguna, penghapusan transaksi melalui aksi geser (swipe-to-delete Dismissible) maupun tombol hapus dilindungi oleh AlertDialog konfirmasi. Operasi penghapusan pada basis data SQLite hanya akan dieksekusi apabila pengguna secara eksplisit menekan tombol 'Hapus'.



Gambar 2.9: Daftar Transaksi Terkini Setelah Transaksi Dihapus

Tangkapan layar di atas memperlihatkan status daftar transaksi setelah transaksi pengeluaran dihapus. Record transaksi terkait terhapus secara permanen dari tabel transactions di SQLite, dan callback penyegaran memicu kalkulasi ulang sehingga saldo dan ringkasan anggaran kembali ke nilai sebelum transaksi tersebut ditambahkan.



Gambar 2.10: Tampilan Antarmuka dalam Mode Gelap (Dark Mode)

Pengguna dapat beralih ke Dark Mode hanya dengan satu ketukan pada ikon tema di AppBar. Antarmuka aplikasi secara instan menyesuaikan seluruh palet warna Material 3 menjadi gelap dengan kontras warna teks dan kartu yang tetap nyaman dibaca pada kondisi pencahayaan rendah. Status preferensi tema disimpan ke SharedPreferences.



Gambar 2.11: Pembuktian Persistensi Tema dan Data Offline Setelah Relaunch

Tangkapan layar ini membuktikan ketahanan persistensi data aplikasi. Perangkat emulator dimatikan koneksi internetnya secara total (offline) dan aplikasi DompetKu dihentikan paksa (force-stop via adb shell am force-stop). Saat aplikasi dibuka kembali dari kondisi dingin (cold start), tema gelap langsung aktif secara otomatis dari SharedPreferences, dan seluruh data saldo serta transaksi dari SQLite tetap utuh sempurna tanpa kehilangan data sedikit pun.

2.2 Persiapan Project dan Package

Pengembangan aplikasi DompetKu dilakukan menggunakan SDK Flutter versi stable yang telah terkonfigurasi pada lingkungan sistem operasi Windows 11 dengan target build Android. Struktur direktori project dirancang modular dan menerapkan prinsip pemisahan tanggung jawab (Separation of Concerns) guna mempermudah pemeliharaan, pengujian, serta keterbacaan kode:

1. Folder lib/models/: Berisi representasi entitas data murni, yaitu category.dart (model kategori transaksi beserta pemetaan toMap dan fromMap) dan transaction_item.dart (model transaksi yang mencakup relasi category_id, tipe, nominal, tanggal, serta atribut join untuk kemudahan visualisasi antarmuka).

2. Folder lib/database/: Berisi database_helper.dart yang mengelola inisialisasi basis data SQLite, eksekusi skema DDL, relasi foreign key, pengindeksan, operasi CRUD asinkron, query agregasi keuangan, serta mekanisme sinkronisasi pembukaan basis data.

3. Folder lib/services/: Berisi modul layanan independen, meliputi theme_preference_service.dart (pengelolaan preferensi tema via SharedPreferences), budget_preference_service.dart (pengelolaan batas anggaran via SharedPreferences), serta currency_helper.dart (utilitas format Rupiah, format tanggal Bahasa Indonesia, dan pemetaan ikon kategori).

4. Folder lib/screens/: Berisi halaman antarmuka pengguna, meliputi transaction_form_screen.dart (layar formulir penambahan dan pengeditan transaksi) serta main.dart yang memuat DompetKuApp, DashboardScreen, dan TransactionsScreen.

5. Folder test/: Berisi rangkaian pengujian otomatis komprehensif, mencakup budget_test.dart (unit test preferensi anggaran), database_test.dart (pengujian transaksi nyata SQLite FFI, batasan nilai, dan integritas referensial), serta widget_test.dart (pengujian responsivitas widget dan navigasi).

Dalam berkas konfigurasi pubspec.yaml, dependensi eksternal dipilih secara cermat sesuai dengan batasan tugas akademik agar aplikasi tetap ringan dan terfokus:

a. sqflite (versi ^2.4.5): Driver basis data SQLite resmi untuk Flutter yang menyediakan akses ke SQLite embedded engine pada Android. Package ini digunakan untuk seluruh penyimpanan data transaksi dan kategori secara terstruktur dan relasional.

b. path (versi ^1.9.1): Utilitas standar Dart untuk memanipulasi dan menyusun path direktori sistem berkas secara aman dan platform-independent saat menentukan lokasi berkas berkas dompetku.db.

c. shared_preferences (versi ^2.5.6): Plugin Flutter untuk membungkus penyimpanan persisten data primitif (key-value store) berbasis SharedPreferences pada Android. Package ini dipakai untuk menyimpan preferensi tema gelap/terang dan nominal batas anggaran bulanan.

d. intl (versi ^0.20.3): Package internasionalisasi dari Dart untuk memformat angka mata uang menjadi format Rupiah standar Indonesia (NumberFormat.currency) serta menyajikan format tanggal kalender lokal (initializeDateFormatting('id_ID')).

2.3 Implementasi Asynchronous Programming

Dalam pengembangan aplikasi mobile, operasi I/O seperti membaca disk, mengakses basis data, atau menulis preferensi memerlukan waktu eksekusi yang

tidak instan. Apabila operasi tersebut dijalankan secara sinkron pada main thread (UI thread), aplikasi akan mengalami freeze (jank) atau terhenti sementara (Not Responding). Untuk mencegah hal tersebut, Dart dan Flutter menyediakan paradigma Asynchronous Programming berbasis event-loop dengan konstruksi Future, async, dan await.

Aplikasi DompetKu menerapkan asynchronous programming secara menyeluruh pada setiap lapisan logika aplikasi, mulai dari inisialisasi basis data hingga interaksi antarmuka pengguna:

2.3.1 Inisialisasi Basis Data dan Sinkronisasi Akses Asinkron

Pembukaan basis data SQLite dilakukan secara asinkron menggunakan openDatabase(). Untuk menghindari masalah concurrent initialization lock (di mana dua query dieksekusi secara bersamaan sebelum proses openDatabase selesai), DatabaseHelper menerapkan mekanisme Future cache synchronization (_opening) sehingga beberapa pemanggilan asinkron yang datang bersamaan akan menunggu instansiasi Future yang sama:

```
// lib/database/database_helper.dart
Future<Database> get database async {
  if (_database != null) return _database!;
  _opening ??= _initDB(databaseName);
  try {
    _database = await _opening!;
    return _database!;
  } finally {
    _opening = null;
  }
}

Future<Database> _initDB(String filePath) async {
  final dbPath = await getDatabasesPath();
  final path = join(dbPath, filePath);

  return await openDatabase(
    path,
    version: databaseVersion,
    onCreate: _createDB,
    onConfigure: (db) async {
      await db.execute('PRAGMA foreign_keys = ON');
    },
  );
}
```

2.3.2 Operasi CRUD dan Agregasi Keuangan Asinkron

Setiap operasi manipulasi data transaksi (insert, update, delete) dideklarasikan dengan kata kunci async dan mengembalikan Future<int>. Sebelum data dikirim ke SQLite, dilakukan validasi asinkron untuk memastikan integritas data. Selain itu, kalkulasi ringkasan saldo, total pemasukan, dan total pengeluaran dilakukan secara asinkron melalui query agregasi SQL:

```
// lib/database/database_helper.dart
Future<int> insertTransaction(TransactionItem transaction) async {
  if (!transaction.amount.isFinite || transaction.amount <= 0) {
    throw ArgumentError('Nominal transaksi harus berupa angka
positif terbatas');
  }
  final category = await getCategoryById(transaction.categoryId);
  if (category == null || category.type != transaction.type) {
    throw ArgumentError('Kategori tidak valid atau tipe tidak
sesuai dengan transaksi');
  }
  final db = await database;
  return await db.insert('transactions', transaction.toMap());
}

Future<Map<String, double>> getFinancialSummary({String?
monthPrefix}) async {
  final db = await database;
  final allTotalsQuery = '''
    SELECT
      COALESCE(SUM(CASE WHEN type = 'income' THEN amount ELSE 0
END), 0) AS total_income,
      COALESCE(SUM(CASE WHEN type = 'expense' THEN amount ELSE 0
END), 0) AS total_expense
    FROM transactions
  ''';
  final allTotals = await db.rawQuery(allTotalsQuery);
  final totalIncome = (allTotals.first['total_income'] as
num).toDouble();
  final totalExpense = (allTotals.first['total_expense'] as
num).toDouble();
  final currentBalance = totalIncome - totalExpense;

  double monthlyIncome = totalIncome;
  double monthlyExpense = totalExpense;

  if (monthPrefix != null && monthPrefix.isNotEmpty) {
    final monthlyQuery = '''
      SELECT
        COALESCE(SUM(CASE WHEN type = 'income' THEN amount ELSE 0
END), 0) AS month_income,
```

```
        COALESCE(SUM(CASE WHEN type = 'expense' THEN amount ELSE 0
END), 0) AS month_expense
      FROM transactions
      WHERE transaction_date LIKE ?
    ''';
    final monthlyTotals = await db.rawQuery(monthlyQuery,
['$monthPrefix%']);
    monthlyIncome = (monthlyTotals.first['month_income'] as
num).toDouble();
    monthlyExpense = (monthlyTotals.first['month_expense'] as
num).toDouble();
  }

  return {
    'currentBalance': currentBalance,
    'totalIncome': totalIncome,
    'totalExpense': totalExpense,
    'monthlyIncome': monthlyIncome,
    'monthlyExpense': monthlyExpense,
  };
}
```

2.3.3 Pembacaan dan Penulisan Preferensi Asinkron

Operasi SharedPreferences bersifat non-blocking dan menggunakan Future. Nilai batas anggaran dan preferensi tema dibaca secara asinkron saat aplikasi dimuat dan disimpan kembali saat pengguna melakukan perubahan:

```
// lib/services/budget_preference_service.dart
class BudgetPreferenceService {
  static const String keyMonthlyBudget = 'dompetku_monthly_budget';

  static Future<double> getMonthlyBudget() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getDouble(keyMonthlyBudget) ?? 0.0;
  }

  static Future<bool> setMonthlyBudget(double amount) async {
    if (!amount.isFinite || amount < 0) {
      throw ArgumentError('Anggaran harus angka terbatas dan tidak
negatif');
    }
    final prefs = await SharedPreferences.getInstance();
    return await prefs.setDouble(keyMonthlyBudget, amount);
  }
}
```

2.3.4 Penanganan State Asinkron dan Feedback Antarmuka

Pada sisi antarmuka pengguna (UI), metode _load() memanggil seluruh layanan data secara paralel dengan penanganan status loading, pengecekan mounted sebelum memanggil setState, serta blok try-catch untuk menangani potensi kesalahan jaringan/disk secara aman:

```
// lib/main.dart (DashboardScreenState)
Future<void> _load() async {
  if (mounted) setState(() { loading = true; error = null; });
  try {
    final now = DateFormat('yyyy-MM').format(DateTime.now());
    final s = await db.getFinancialSummary(monthPrefix: now);
    final l = await db.getAllTransactions(limit: 5);
    final b = await BudgetPreferenceService.getMonthlyBudget();
    if (!mounted) return;
    setState(() {
      summary = s;
      latest = l;
      budget = b;
      loading = false;
    });
  } catch (e) {
    if (mounted) {
      setState(() {
        error = 'Data tidak dapat dimuat: $e';
        loading = false;
      });
    }
  }
}
```

2.4 Implementasi Data Persistence

Persistensi data (Data Persistence) merupakan kemampuan sistem untuk menyimpan status dan informasi secara persisten sehingga data tidak hilang saat aplikasi ditutup, dimatikan dari latar belakang, atau saat perangkat direstart. Sesuai dengan pedoman tugas Ujian Tengah Semester dan materi perkuliahan BAB 3, aplikasi DompetKu menerapkan dua teknik persistensi data lokal yang memiliki karakteristik berbeda:

2.4.1 Teknik 1: SQLite Database (Relational Data Persistence)

SQLite digunakan untuk menyimpan entitas data transaksional yang memiliki relasi terstruktur. Basis data lokal diberi nama dompetku.db versi 1. Skema basis data terdiri dari dua tabel utama yang saling terhubung melalui relasi One-to-Many:

1. Tabel categories: Menyimpan daftar kategori transaksi dengan kolom id (INTEGER PRIMARY KEY AUTOINCREMENT), name (TEXT NOT NULL), type (TEXT NOT NULL), icon (TEXT NOT NULL), dan color (INTEGER NOT NULL).

2. Tabel transactions: Menyimpan catatan transaksi keuangan dengan kolom id (INTEGER PRIMARY KEY AUTOINCREMENT), category_id (INTEGER NOT NULL), type (TEXT NOT NULL), amount (REAL NOT NULL), transaction_date (TEXT NOT NULL), description (TEXT), dan created_at (TEXT NOT NULL).

Penerapan integritas referensial ditegakkan secara ketat melalui perintah PRAGMA foreign_keys = ON dan klausa FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE RESTRICT. Hal ini menjamin bahwa kategori yang masih dirujuk oleh sebuah transaksi tidak dapat dihapus secara sembarangan, sehingga mencegah terjadinya kondisi data anomali (orphaned records).

Selain itu, dibuat indeks khusus idx_transactions_date pada kolom transaction_date DESC untuk mempercepat eksekusi query pemfilteran riwayat transaksi bulanan:

```
// lib/database/database_helper.dart (_createDB)
await db.execute('''
  CREATE TABLE categories (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    type TEXT NOT NULL,
    icon TEXT NOT NULL,
    color INTEGER NOT NULL
  )
''');

await db.execute('''
  CREATE TABLE transactions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    category_id INTEGER NOT NULL,
    type TEXT NOT NULL,
    amount REAL NOT NULL,
    transaction_date TEXT NOT NULL,
    description TEXT,
    created_at TEXT NOT NULL,
    FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE
RESTRICT
  )
''');
```

```
await db.execute('''
  CREATE INDEX idx_transactions_date ON transactions
(transaction_date DESC)
''');
```

2.4.2 Teknik 2: SharedPreferences (Key-Value Data Persistence)

SharedPreferences digunakan untuk menyimpan data konfigurasi aplikasi dan preferensi pengguna yang bersifat ringan berupa pasangan kunci dan nilai (key-value pair). Pada aplikasi DompetKu, SharedPreferences menyimpan dua informasi penting:

1. dompetku_is_dark_mode (tipe boolean): Menyimpan status apakah pengguna memilih mode gelap (true) atau mode terang (false). Nilai ini langsung dibaca sebelum widget MaterialApp dibentuk pada fungsi main() sehingga antarmuka tidak mengalami kedipan (flicker) warna saat peluncuran aplikasi.

2. dompetku_monthly_budget (tipe double): Menyimpan nominal batas pengeluaran bulanan yang ditetapkan pengguna. Nilai ini menjadi acuan persentase pada indikator kemajuan dashboard.

Kedua teknik persistensi ini saling melengkapi: SQLite menangani kebutuhan data relasional yang dinamis dan berukuran besar, sedangkan SharedPreferences menangani konfigurasi status aplikasi yang sederhana dan cepat diakses.

2.5 Pengujian dan Hasil Implementasi

Pengujian sistem dilakukan secara bertingkat dan berbasis bukti eksekusi riil, mencakup analisis kode statis, pengujian unit dan widget otomatis, kompilasi paket instalasi Android (APK), serta pengujian fungsional dan persistensi end-to-end langsung pada Android Emulator.

2.5.1 Hasil Analisis Statis dan Pengujian Otomatis

a. flutter analyze: Perintah analisis kode statis dijalankan pada direktori project dompetku_uts dan menghasilkan status bersih sempurna tanpa kesalahan, tanpa peringatan, dan tanpa rekomendasi perbaikan (No issues found! ran in 1.8s).

b. flutter test: Rangkaian 7 automated test suites yang mencakup pengujian validasi nominal, operasi CRUD SQLite nyata via sqflite_common_ffi, penegakan batasan Foreign Key RESTRICT, persistensi SharedPreferences, serta uji render responsif pada resolusi 320x568 dp dan 412x915 dp berhasil lulus 100% (7 passed, 0 failed, ran in 00:03).

c. flutter build apk --debug: Proses kompilasi paket aplikasi Android debug berhasil diselesaikan dan menghasilkan berkas instalasi app-debug.apk berukuran 154 MB pada lokasi build/app/outputs/flutter-apk/app-debug.apk.

2.5.2 Tabel Hasil Pengujian Fungsional dan Persistensi Nyata

Berikut adalah rangkuman matriks pengujian fungsional dan pembuktian persistensi yang diuji secara langsung pada emulator Android (API level 34) dalam alur penggunaan nyata:

No	Skenario Pengujian	Langkah Pengujian	Hasil yang Diharapkan	Hasil Aktual & Status
1	Peluncuran Awal Aplikasi (Fresh Start)	Buka aplikasi DompetKu setelah instalasi bersih pada emulator.	Aplikasi terbuka langsung tanpa login; saldo Rp 0; kategori awal ter-seed di SQLite.	Berhasil. Tampil dashboard awal bersih sesuai Gambar 2.1.
2	Pengaturan Batas Anggaran	Tekan ikon tabungan di AppBar, masukkan nominal Rp 3.000.000, lalu tekan Simpan.	Nilai tersimpan di SharedPreferences; dashboard menampilkan batas anggaran Rp 3.000.000.	Berhasil. Anggaran terpasang dan tersimpan persisten (Gambar 2.2).
3	Pencatatan Transaksi Pemasukan	Buka form (+ Transaksi), pilih tipe Pemasukan, isi nominal 5000000, kategori Gaji, lalu Simpan.	Data tersimpan di tabel transactions; saldo dan pemasukan bulan ini menjadi Rp 5.000.000.	Berhasil. Saldo dan daftar transaksi terbaru teragregasi (Gambar 2.4).
4	Pencatatan Transaksi Pengeluaran	Buka form (+ Transaksi), pilih tipe Pengeluaran, isi 150000, kategori Makanan, lalu Simpan.	Saldo berkurang menjadi Rp 4.850.000; progress bar anggaran terisi 5% (sisa Rp 2.850.000).	Berhasil. Agregasi pengeluaran dan anggaran terhitung realtime (Gambar 2.5).
5	Penyaringan Riwayat Transaksi	Buka halaman Transaksi, pilih filter tipe Pengeluaran atau filter kategori tertentu.	Daftar transaksi hanya menampilkan data yang sesuai dengan kriteria filter aktif.	Berhasil. Filter dropdown menyaring data SQL secara akurat (Gambar 2.7).
6	Pengeditan Transaksi	Ketuk salah satu transaksi, ubah nominal atau catatan, lalu tekan Perbarui Transaksi.	Data transaksi diperbarui di SQLite; saldo dashboard otomatis disesuaikan.	Berhasil. Record terupdate dengan id dan tanggal yang konsisten (Gambar 2.6 & 2.7).
7	Penghapusan Transaksi	Geser item transaksi atau tekan aksi hapus, lalu konfirmasi pada dialog pop-up.	Muncul dialog konfirmasi; jika disetujui, record terhapus dan saldo diperbarui.	Berhasil. Dialog konfirmasi muncul dan data terhapus aman (Gambar 2.8 & 2.9).
8	Uji Foreign Key Constraint	Jalankan query penghapusan kategori pada kategori yang sedang digunakan transaksi.	Operasi ditolak oleh SQLite engine dengan melempar DatabaseException (FK RESTRICT).	Berhasil. Terbukti lulus pada automated test database_test.dart baris 37.
9	Pengalihan Tema Gelap / Terang	Tekan tombol toggle tema pada AppBar untuk mengaktifkan Dark Mode.	Antarmuka berubah menjadi tema gelap; preferensi disimpan	Berhasil. Palet warna Material 3 berubah kontras dan nyaman (Gambar 2.10).

			ke SharedPreferences.	
10	Uji Cold Relaunch Aplikasi	Hentikan paksa aplikasi via 'am force-stop', lalu buka kembali aplikasi dari launcher.	Aplikasi terbuka dengan tema gelap aktif; saldo dan transaksi tetap utuh.	Berhasil. Persistensi SharedPreferences dan SQLite terbukti utuh (Gambar 2.11).
11	Uji Akses Offline Total	Matikan koneksi WiFi dan Data Seluler pada emulator, lalu operasikan seluruh fitur.	Semua fitur transaksi, saldo, riwayat, dan tema berjalan normal tanpa internet.	Berhasil. Aplikasi 100% mandiri tanpa ketergantungan cloud/jaringan (Gambar 2.11).
12	Uji Responsivitas Tata Letak	Uji pada layar 320dp & 412dp, serta buka keyboard virtual pada form transaksi.	Tidak terjadi RenderFlex Overflow; form dapat digulir bebas di atas keyboard.	Berhasil. Lulus uji widget automated dan pengujian emulator tanpa error.

Tabel 2.1: Matriks Hasil Pengujian Fungsional dan Persistensi Aplikasi DompetKu

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan proses perancangan, implementasi, dan pengujian yang telah dilaksanakan pada pengembangan aplikasi DompetKu untuk tugas Ujian Tengah Semester Pemrograman Multiplatform, dapat diambil beberapa kesimpulan pokok:

1. Aplikasi DompetKu berhasil dibangun menggunakan framework Flutter dan bahasa Dart sebagai aplikasi manajemen kas pribadi harian yang mandiri (standalone) dengan prinsip offline-first, tanpa memerlukan backend eksternal, Firebase, maupun server XAMPP.

2. Konsep Asynchronous Programming menggunakan Future, async, dan await telah diimplementasikan secara nyata pada seluruh operasi I/O, meliputi pembukaan basis data, transaksi CRUD, kalkulasi agregasi finansial, serta pembacaan/penulisan preferensi lokal, sehingga antarmuka pengguna tetap responsif dan bebas dari jank.

3. Dua teknik Data Persistence lokal yang berbeda berhasil diterapkan secara harmonis: SQLite (melalui package sqflite dan path) menangani struktur data relasional transaksi dan kategori dengan penegakan integritas Foreign Key ON DELETE RESTRICT, sedangkan SharedPreferences menangani data konfigurasi tema gelap/terang dan batas anggaran bulanan.

4. Pengujian bertingkat membuktikan keandalan sistem secara objektif: kode sumber bersih dari kesalahan analisis statis (flutter analyze 0 issues), seluruh 7 automated test suites lulus 100%, kompilasi APK debug berhasil diselesaikan, dan pengujian emulator membuktikan data tetap utuh sempurna setelah aplikasi dihentikan paksa (force-stop) dan dibuka kembali dalam kondisi offline total.

3.2 Saran

Untuk pengembangan dan penyempurnaan aplikasi DompetKu pada tahap selanjutnya, disarankan beberapa peningkatan fitur:

1. Penambahan Fitur Ekspor Laporan Keuangan: Menyediakan kemampuan ekspor riwayat transaksi ke dalam format CSV atau dokumen PDF lokal agar pengguna dapat mencetak dan mengarsipkan pembukuan kas bulanan.

2. Visualisasi Grafik Statistik: Menambahkan grafik diagram lingkaran (Pie Chart) atau diagram batang untuk menampilkan proporsi pengeluaran per kategori secara visual dan interaktif.

3. Fitur Pengingat Catat Harian (Local Notifications): Memanfaatkan modul notifikasi lokal untuk mengingatkan pengguna mencatat transaksi setiap malam secara otomatis tanpa memerlukan server internet.

DAFTAR PUSTAKA

Flutter Documentation. (2024). Asynchronous Programming: Futures, async, await. Google Open Source. https://dart.dev/codelabs/async-await

Flutter Documentation. (2024). Persist data with SQLite. Google Developer Docs. https://docs.flutter.dev/cookbook/persistence/sqlite

Flutter Documentation. (2024). Store key-value data on disk using shared_preferences. Google Developer Docs. https://docs.flutter.dev/cookbook/persistence/key-value

Google. (2023). Material Design 3 Guidelines for Android and Flutter. Google Design. https://m3.material.io/

Intl Package Documentation. (2024). Internationalization and localization support for Dart and Flutter. Dart Packages Repository. https://pub.dev/packages/intl

SQLite Consortium. (2024). SQLite Foreign Key Support and ACID Transactions. SQLite Official Documentation. https://www.sqlite.org/foreignkeys.html

Sqflite Package Documentation. (2024). SQLite plugin for Flutter: DatabaseHelper and raw queries. Dart Packages Repository. https://pub.dev/packages/sqflite

Tim Dosen Pemrograman Multiplatform. (2024). Modul Praktikum BAB 3: Asynchronous Programming dan Data Persistence pada Flutter. Jurusan Teknik Informatika, Fakultas Sains dan Teknologi, UIN Maulana Malik Ibrahim Malang.